

point is that the repository interface belongs to the domain layer whereas its implementation belongs to infrastructure. Anything that is related to the infrastructure layer is expected to be pluggable.

And finally, the data transfer objects, as the name suggests, are special kinds of objects that are meant for request and response payloads. They do not belong to the domain layer; instead, they are useful in the application layer. But, what is the need for DTOs? Why can't entities and VOs be used for transferring the data?

The reason is self-explanatory. The entities and value objects are domain objects. A domain object is useful and relevant within its own domain or sub-domain. Exposing the domain model outside of the domain creates a needless coupling between the domain and the outside world. For instance, the name of a customer might be modelled as a value object within the domain model with *FirstName* and *LastName* as the fields. However, for the outsiders it is immaterial. They may see the name as just a character sequence. A DTO does the job of converting a name object into a string and vice versa.

Illustration

Let us look at some examples to understand the domain-driven design patterns. These examples have nothing to do with any specific programming language. Consider them as a pseudo-language to convey the idea.

Following are the classes *Order* and *OrderManagement*:

```
class Order {
    private double amount;
    private Date date;
    private String status;
    ....
    getAmount();
    getDate();
    getStatus();

    setAmount(double);
    setDate(Date date);
    setStatus(String status);
}

class OrderManagement {
    public void cancel(Order order) {
        apply business rules;
        order.setStatus("cancelled");
    }
    ....
}
```

The problem with the above design is that the *status* property of *Order* is being manipulated by an outsider *OrderManagement*. Where is the encapsulation? This is more like global variables and global functions with a bit of sugar-coating in the form of setters and getters. The status of the *Order* must be handled by the *Order* itself since it is its state property. The following is a better version of the design:

```
class Order {
    private double amount;
    private Date date;
    private String status;
    ....
    getAmount();
    getDate();
    getStatus();

    setAmount(double);
    setDate(Date date);

    public void cancel() {
        apply business rules
        status = "cancelled";
    }
}
```

The above *Order* class does not expose any direct setters. However, it should not be understood as if setters are bad. The object is still mutable, but through more meaningful methods like *cancel()*.

The next enhancement to the above class is to add an identity. Remember that it is not a technically-driven design decision. In the e-commerce domain, every order is given a unique identity. We are merely mapping to the real world.

```
class Order {
    private int id;
    private double amount;
    private Date date;
    private String status;
    ....
    getId();
    getAmount();
    getDate();
    getStatus();

    setId(int id);
    setAmount(double);
    setDate(Date date);

    public void cancel() {
```